

Aufgabe 3: Tobis Turnier

Team-ID: 00048

Team: `while(!success){try();}`

Bearbeiter/-innen dieser Aufgabe:
Rui Zhang

21. November 2020

Inhaltsverzeichnis

Lösungsidee.....	1
Umsetzung.....	2
Beispiele.....	2
Quellcode.....	3

Lösungsidee

Um Tobis Zufallsexperiment umzusetzen, soll die `Random()`-Bibliothek von Java angewendet werden. Hierbei wird eine (pseudo)zufällige ganzzahl zwischen 0 (inklusive) und der Summe der Spielstärken der zwei Spieler (exklusiv), die gegeneinander antreten, erzeugt. Wenn also die generierte Zahl kleiner ist als die Spielstärke des ersten Spielers, dann hat dieser gewonnen, und wenn das nicht der Fall ist, dann hat der zweite Spieler gewonnen.

Im Folgenden werden die Lösungsideen für die einzelnen Turniervarianten erklärt. Dabei ist zu beachten, dass jede Turniervariante viele Male wiederholt werden soll, und die Anzahl der Turniersiegen des besten Spielers zurückgegeben wird. Um diese zu bestimmen wird der Sieger der einzelnen Turnierwiederholungen mit dem Spieler mit der höchsten Spielstärke verglichen. Falls diese gleich sind, dann wird die Anzahl der Turniersiegen des besten Spielers inkrementiert.

Liga:

Um die Turniervariante „Liga“ umzusetzen, sollen alle Kombinationen zweier gegeneinander spielenden Spielern durchgegangen werden. Jedes mal wird die Lösungsidee für Tobis Zufallsexperiment angewendet, um den Sieger dieser Runde zu ermitteln. Nachdem alle Runden durchgeführt wurden, wird die Anzahl der Siege des spielstärksten Spielers zurückgegeben.

K.O.:

Die Turniervariante „K.O.“ soll mithilfe eines rekursiven Algorithmus umgesetzt werden. Eine Methode bekommt eine Liste, welche in zwei Hälften geteilt wird. Die beiden Hälften werden in dieselbe Methode eingesetzt und so weiter. Das geht so lange, bis es in einer Rekursion nur noch zwei Spieler gibt. Diese treten gegeneinander an und der Sieger wird zurückgegeben. Die zurückgegebenen Sieger treten dann wiederum gegeneinander an und der Sieger aus den Siegern der vorherigen Runde wird zurückgegeben. Das geht weiter bis der Algorithmus beim ersten Aufruf dieser Methode angekommen ist, wo dann der endgültige Sieger bestimmt wird. Somit wird eine Wiederholung der Turniervariante K.O. simuliert.

K.O. * 5:

Diese Turniervariante wird genauso wie die Turniervariante K.O. umgesetzt. Diesmal wird Tobis Zufallsexperiment in jeder Runde fünf mal wiederholt und der Spieler mit der höchsten Anzahl von Siegen in dieser Runde wird zurückgegeben.

Umsetzung

Im eigentlichen Programm gibt es zwei Klassen.

Die Spielerobjekte stellen die einzelnen Spieler dar und speichern Spielernummer, Spielstärke und die Anzahl der Siege. Letztere ist NICHT die insgesamt Anzahl der Siege nach allen Wiederholungen einer Turniervariante. Stattdessen ist die Anzahl der Siege beispielsweise die in einem Liga-Turnier oder die Anzahl der Siege in einer Runde von K.O * 5. Deshalb wird dieses Attribut nach jedem Liga-Turnier oder am Anfang jeder K.O * 5 Runde zurückgesetzt (auf 0).

Die RNG Klasse des Programms speichert die Anzahl der Wiederholungen und liest eine Datei mit dem Format der Beispiele auf der BWINF Webseite ein und erstellt daraus eine Liste von allen Spielern. Währenddessen wird auch der stärkste Spieler ermittelt.

Die einzelnen Turniervarianten wurden als separate Methoden in der RNG Klasse implementiert. Diese geben die Anzahl der Siege des spielstärksten Spielers zurück und werden in einer Methode `turnierVariantenTest()` aufgerufen. Die Anzahl der Siege von den verschiedenen Turniervarianten werden in der Methode verglichen und dem Tobi wird die Turniervariante mit der höchsten Anzahl von Siegen des spielstärksten Spielers über mehreren Wiederholungen empfohlen.

In der Main-Methode des Programms wird zunächst eine Anleitung angegeben. Der Benutzer gibt eine Beispieldatei (0, 1, 2, 3 oder 4) sowie die Anzahl der Wiederholungen, die jede Simulation durchführen soll, an. Die Lösung dieses Beispiels wird dann angegeben.

Beispiele

Die Beispiele von der BWINF Webseite wurden angewendet.

Beispiel 0 stellte eine sehr ausgewogene Verteilung aller Spielstärken dar. Hier war die Empfehlung für Tobi KO mal 5. Beispiel 3 aber, war genau das Gegenteil davon. Alle Spieler hatten eine Spielstärke von 95 außer ein Spieler, welcher die Spielstärke 100 besaß. Hier war die Empfehlung Liga. Bei Beispiel 1 empfahl das Programm die Turniervariante KO mal 5. Bei Beispiel 2 wieder die Turniervariante Liga.

Beispiel 2 und Beispiel 3 waren ähnlich im Sinne der Spielstärken. Bei beiden Beispielen waren die Spielstärken nur in einem sehr kleinen Intervall. Beispiel 0 und 1 besaßen aber größere Intervalle. In Beispiel 0 gab es Spielstärken von 0 bis 100 und in Beispiel 1 von 10 bis 100. Es scheint so, als sei die KO mal 5 Variante mehr für große Spielstärkenintervalle geeignet und Liga für kleinere.

KO als Turniervariante wird bei keinem der Beispiele empfohlen, außer bei einer sehr kleinen Anzahl von Wiederholungen. Aber sogar dann kommt es nur zufällig zu dieser Empfehlung, denn bei wiederholtem Laufen des Programms mit demselben Beispiel und mit derselben kleinen Anzahl von Wiederholungen erscheinen oft andere Turniervarianten als Empfehlung. Dies war mehr oder weniger zu erwarten, denn schließlich startet der spielstärkste Spieler „ganz unten“ und muss sich „hocharbeiten“ wie in der Abbildung auf der BWINF Webseite. Hierbei braucht es nur zu einer verunglückten Runde kommen und schon ist der spielstärkste Spieler, welcher eigentlich gewinnen sollte, aus dem Spiel raus und kann nicht mehr gewinnen.

Quellcode

```
public class RNG {  
    private final int anzahlWiederholungen;  
    private final ArrayList<Spieler> spieler;  
    private Spieler staerksterSpieler;  
  
    public RNG(String path, int anzahlWiederholungen) throws IOException, NullPointerException  
    {  
        this.anzahlWiederholungen = anzahlWiederholungen;  
        this.staerksterSpieler = new Spieler(0, 0);  
        this.spieler = new ArrayList<>();  
  
        BufferedReader reader = new BufferedReader(new  
InputStreamReader(getClass().getResourceAsStream(path), StandardCharsets.UTF_8));  
        reader.readLine();  
  
        int i = 0;
```

```
String zeile;
while ((zeile = reader.readLine()) != null) {
    i++;
    Spieler jetzigerSpieler = new Spieler(i, Integer.parseInt(zeile));
    this.spieler.add(jetzigerSpieler);
    if (jetzigerSpieler.getSpielStaerke() > this.staerksterSpieler.getSpielStaerke()) {
        this.staerksterSpieler = jetzigerSpieler;
    }
}

}

private void siegeReset() {
    for (Spieler spieler : this.spieler) {
        spieler.setAnzahlSiege(0);
    }
}

private Spieler ziehen(Spieler ersterSpieler, Spieler zweiterSpieler) {
    int gezogen = new Random().nextInt(ersterSpieler.getSpielStaerke() +
    zweiterSpieler.getSpielStaerke() + 1);
    return gezogen < ersterSpieler.getSpielStaerke() ? ersterSpieler : zweiterSpieler;
}

private int liga() {
    int anzahlGewinneStaerksterSpieler = 0;
    for (int i=0; i<this.anzahlWiederholungen; i++) {
        Spieler meisteGewinne = this.spieler.get(0);
        int startIndex = 0;
```

```
for (Spieler ersterSpieler : this.spieler) {
    for (Spieler zweiterSpieler : this.spieler.subList(startIndex, this.spieler.size())) {
        Spieler gewinner = this.ziehen(ersterSpieler, zweiterSpieler);
        gewinner.siegeInkrement();

        if ((gewinner.getAnzahlSiege() > meisteGewinne.getAnzahlSiege()) ||
            gewinner.getAnzahlSiege() == meisteGewinne.getAnzahlSiege() &&
            gewinner.getNummer() < meisteGewinne.getNummer())
            meisteGewinne = gewinner;
    }

    startIndex++;
}
this.siegeReset();

if (meisteGewinne == this.staerksterSpieler)
    anzahlGewinneStaerksterSpieler++;
}
return anzahlGewinneStaerksterSpieler;
}
```

```
private int KO() {
    int anzahlGewinneStaerksterSpieler = 0;
    for (int i=0; i<this.anzahlWiederholungen; i++) {

        Collections.shuffle(this.spieler);
        Spieler gewinner = KORunde(this.spieler);

        if (gewinner == this.staerksterSpieler)
```

```
        anzahlGewinneStaerksterSpieler++;

    }

    return anzahlGewinneStaerksterSpieler;
}

private Spieler KORunde(List<Spieler> spieler) {
    if (spieler.size() != 2)
        return this.ziehen(
            this.KORunde(spieler.subList(0, spieler.size() / 2)),
            this.KORunde(spieler.subList(spieler.size() / 2, spieler.size())));
    else
        return this.ziehen(spieler.get(0), spieler.get(1));
}

private int KOMal5() {
    int anzahlGewinneStaerksterSpieler = 0;
    for (int i=0; i<this.anzahlWiederholungen; i++) {

        Collections.shuffle(this.spieler);
        Spieler gewinner = KOMal5Runde(this.spieler);

        if (gewinner == this.staerksterSpieler)
            anzahlGewinneStaerksterSpieler++;

    }

    return anzahlGewinneStaerksterSpieler;
}
```

```
private Spieler KOMal5Runde(List<Spieler> spieler) {

    Spieler ersterSpieler, zweiterSpieler;
    if (spieler.size() != 2) {
        ersterSpieler = this.KOMal5Runde(spieler.subList(0, spieler.size() / 2));
        zweiterSpieler = this.KOMal5Runde(spieler.subList(spieler.size() / 2, spieler.size()));
    } else {
        ersterSpieler = spieler.get(0);
        zweiterSpieler = spieler.get(1);
    }

    ersterSpieler.setAnzahlSiege(0);
    zweiterSpieler.setAnzahlSiege(0);

    for (int i=0; i<5; i++) {
        this.ziehen(ersterSpieler, zweiterSpieler).siegInkrement();
    }

    return ersterSpieler.getAnzahlSiege() > zweiterSpieler.getAnzahlSiege() ? ersterSpieler :
    zweiterSpieler;

}

public void turnierVariantenTest() {

    int richtigeLigaSiege = this.liga();
    int richtigeKOSiege = this.KO();
    int richtigeKOMal5Siege = this.KOMal5();
    HashMap<Integer, String> siegeZuTurnierVariante = new HashMap<Integer, String>(){
        put(richtigeLigaSiege, "Liga");
    }
}
```

```
        put(richtigeKOSiege, "KO");  
        put(richtigeKOMal5Siege, "KO*5");  
    };
```

```
    StringBuilder ausgabe = new StringBuilder();  
    ausgabe  
        .append("Simulationsergebnisse:\n")  
        .append("Liga: spielstärkster Spieler hat ").append(richtigeLigaSiege).append(" mal aus  
").append(this.anzahlWiederholungen).append(" gewonnen\n")  
        .append("KO: spielstärkster Spieler hat ").append(richtigeKOSiege).append(" mal aus  
").append(this.anzahlWiederholungen).append(" gewonnen\n")  
        .append("KO*5: spielstärkster Spieler hat ").append(richtigeKOMal5Siege).append(" mal  
aus ").append(this.anzahlWiederholungen).append(" gewonnen\n")  
        .append("Empfehlung für Tobi:  
").append(siegeZuTurnierVariante.get(Math.max(richtigeLigaSiege, Math.max(richtigeKOSiege,  
richtigeKOMal5Siege))));  
  
    JOptionPane.showMessageDialog(null, ausgabe);  
}  
}
```

```
public class Spieler {  
    private final int nummer;  
    private final int spielStaerke;  
    private int anzahlSiege;  
  
    public Spieler(int nummer, int spielStaerke) {  
        this.nummer = nummer;  
        this.spielStaerke = spielStaerke;  
        this.anzahlSiege = 0;  
    }  
}
```

```
public int getAnzahlSiege() { return anzahlSiege; }  
public int getNummer() { return nummer; }  
public int getSpielStaerke() { return spielStaerke; }  
  
public void siegeInkrement() { anzahlSiege++; }  
public void setAnzahlSiege(int anzahl) { anzahlSiege = anzahl; }  
}
```